TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

ING. JORGE ARUN ZAMBRANO CEDEÑO, MG.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: BAILON LAZ JULEISSY CAROLINA

PARALELO: ¨A¨

SEMESTRE: QUINTO

ACTIVIDAD: Estudio de caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad ____

Actividad # ___1___

Estudio de caso - Sistema de Control de Rescate y Adopción Animal

1. **Planteamiento del problema**

Actualmente refugios y organizaciones dedicadas al rescate animal enfrentan desafío de llevar un control eficiente de los ingresos de animales en situación de vulnerabilidad o emergencia. La falta de herramientas centralizadas provoca que la información sobre las personas responsables de la atención (voluntarios y veterinarios)y el seguimiento de cada caso (rescatado, en tratamiento, adopción)se maneje de forma manual o desorganizada la dispersión de datos hace que se dificulte la consulta de información, retrasa la toma de decisiones clínicas y dificulta la adopción y rehabilitación.

Por esta razón, se propone diseñar el Sistema de Control de Rescate y Adopción Animal (SCRAA), una solución informática orientada a centralizar la información de los animales rescatados, registrar los casos, clasificar las especies, asignar responsables y controlar los cambios de estado. El diseño también contempla un mecanismo de notificaciones para informar al personal sobre nuevos casos.

La arquitectura se organizará mediante patrones de diseño que permitan separar responsabilidades, reutilizar componentes y facilitar el mantenimiento del sistema.

1.2 Objetivos Generales

Diseñar la arquitectura lógica de un sistema de control de rescate y adopción animal mediante diagramas UML, patrones de diseño de software y el flujo de trabajo Gitflow, con el propósito de organizar la gestión de casos y facilitar el mantenimiento y la evolución de la solución informática.

1.3 Objetivos Específicos

- Identificar los actores, necesidades y requerimientos funcionales y no funcionales del sistema.
- Representar las funcionalidades y relaciones principales mediante diagramas UML.
- Aplicar los patrones Singleton, Observer, Factory Method y MVC, justificando su utilidad dentro de la arquitectura propuesta.
- Organizar el desarrollo mediante un repositorio GitHub con ramas y commits que permitan mantener un historial de cambios.
- Documentar la arquitectura lógica y las decisiones de diseño para facilitar futuras mejoras.

1.4 Alcance Del Sistema

El sistema permitirá registrar casos de rescate, almacenar los datos básicos de los animales, clasificarlos por especie, asignar voluntarios o veterinarios, actualizar los estados de atención y gestionar el proceso de adopción. También contemplará notificaciones al personal asignado y un historial de cambios para consultar la evolución de cada caso.

El alcance de esta actividad se concentra en el diseño lógico y en la demostración de los patrones y del flujo Gitflow. La implementación completa de una plataforma web, la integración con servicios externos y la gestión real de adopciones podrán desarrollarse en etapas posteriores

1.5 Actores Principales

Ciudadano o reportante: comunica el hallazgo de un animal que necesita ayuda y proporciona los datos disponibles sobre su ubicación y condición.

Voluntario o veterinario: recibe asignaciones, consulta los casos a su cargo y registra las acciones de atención correspondientes. El veterinario puede documentar la atención clínica.

Administrador del refugio: supervisa los casos, registra información, asigna responsables, controla los estados y coordina el proceso de adopción.

1.6 Requerimiento Funcionales

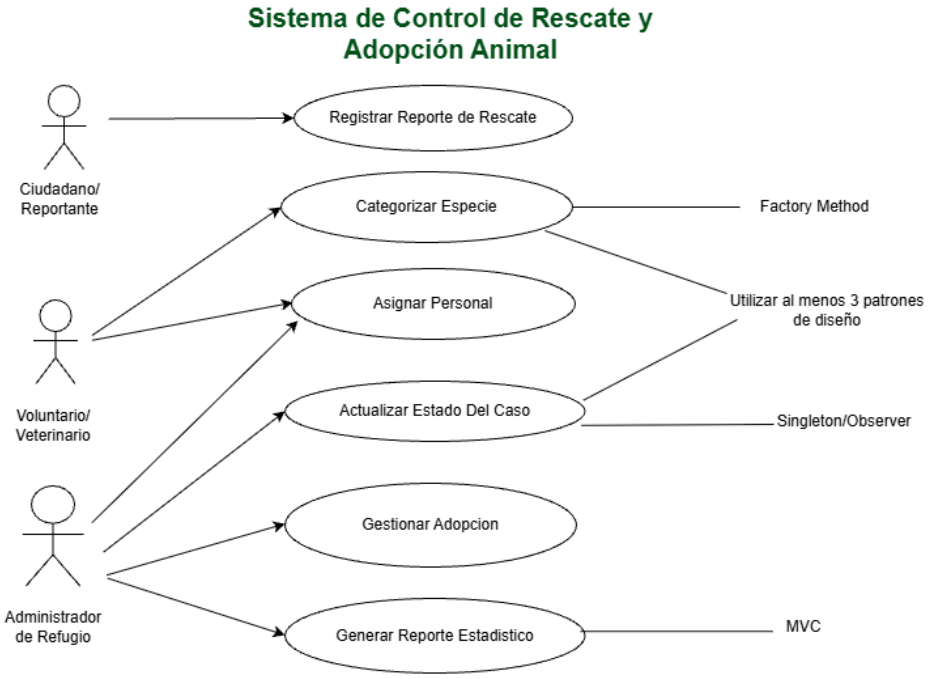
- ✓ RF01. Registrar un nuevo caso de rescate.
- ✓ RF02. Registrar información básica del animal, como nombre provisional, especie, descripción, ubicación y fecha del hallazgo.
- ✓ RF03. Clasificar al animal como perro, gato o fauna silvestre.
- ✓ RF04. Crear un expediente específico según la especie.
- ✓ RF05. Asignar un voluntario o veterinario responsable.
- ✓ RF06. Consultar los casos registrados y sus estados.
- ✓ RF07. Actualizar el estado de un caso según la atención realizada.
- ✓ RF08. Notificar al personal correspondiente cuando se registre un nuevo caso.
- ✓ RF09. Registrar los cambios importantes realizados en cada caso.
- ✓ RF10. Gestionar el paso de un animal disponible para adopción a adoptado, registrando los datos necesarios del proceso.

1.7 Requerimiento No Funcionales

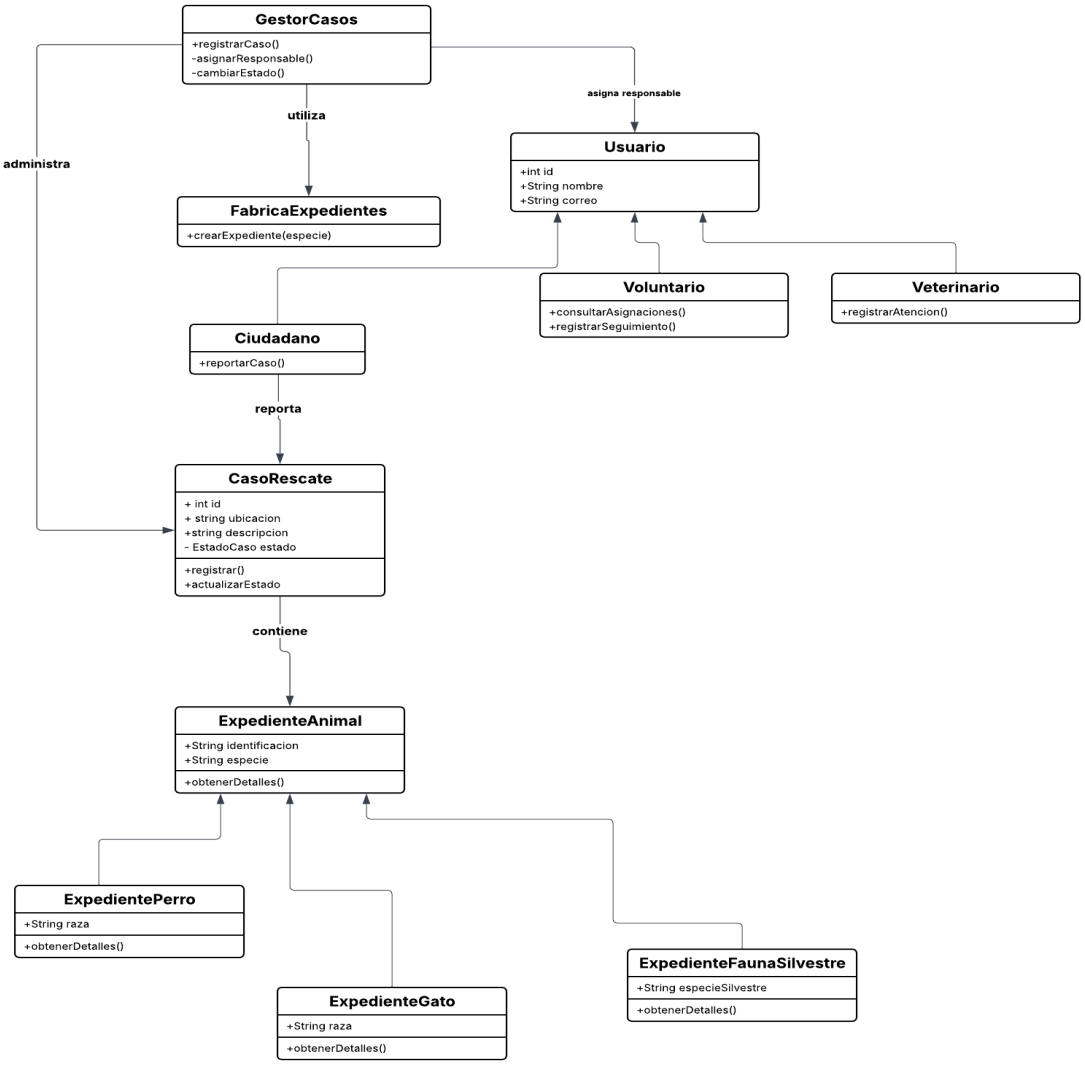
- RNF01. Mantenibilidad: el sistema deberá organizar sus responsabilidades en componentes separados para facilitar las modificaciones.
- RNF02. Escalabilidad: la arquitectura deberá permitir incorporar nuevas especies y funcionalidades sin modificar innecesariamente los componentes existentes.
- RNF03. Reutilización: las funcionalidades comunes deberán aprovecharse en los distintos tipos de expedientes.

- RNF04. Trazabilidad: los cambios del diseño y del código deberán registrarse en Git mediante commits identificables.
- RNF05. Seguridad: el acceso a las operaciones deberá diferenciar los permisos de los usuarios según su función.
- RNF06. Integridad: los cambios de estado y las asignaciones deberán validarse para evitar información inconsistente.
- RNF07. Disponibilidad y confiabilidad: el sistema deberá procurar que los registros puedan consultarse y mantenerse sin pérdida de información.

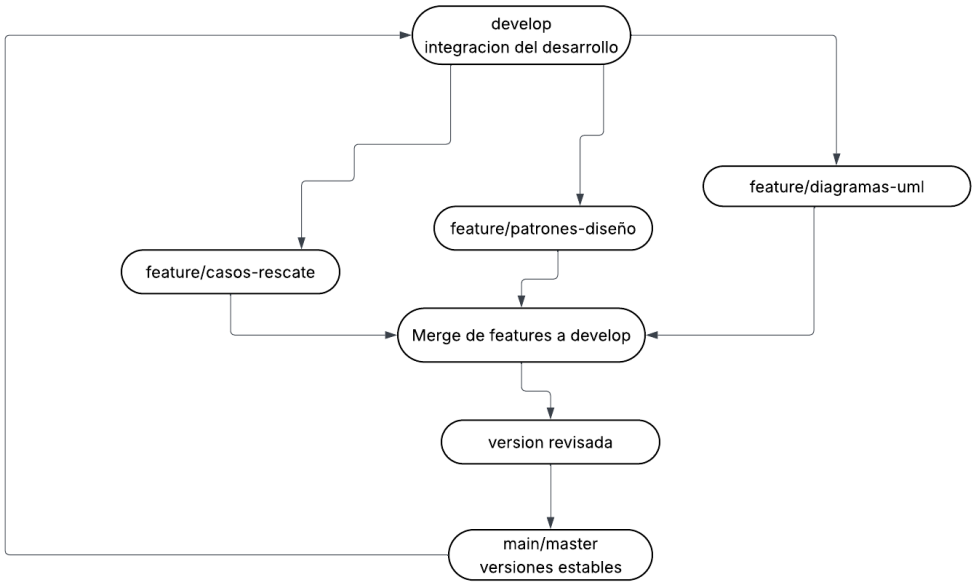
2. Crear Un Diagrama De Caso



3.Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



4.Diseñar la arquitectura general del sistema integrando el flujo de versiones.



La arquitectura general del Sistema de Control de Rescate y Adopción Animal (SCRAA) se organiza en capas para separar las responsabilidades de cada componente. La capa de presentación permite a los usuarios interactuar con el sistema mediante formularios y listados. El controlador recibe las solicitudes y las comunica a la lógica de negocio, donde se coordinan el registro de los casos, la creación de expedientes y las notificaciones al personal correspondiente. Finalmente, la capa de persistencia permite almacenar y consultar los datos de los animales, los responsables y el historial de los casos.

El diseño incorpora el patrón MVC para separar la presentación de la lógica de negocio, Factory Method para crear expedientes según la especie, Singleton para proporcionar un punto de acceso compartido al gestor central y Observer para comunicar la existencia de nuevos rescates a los observadores registrados.

Para organizar el desarrollo se propone Gitflow, utilizando main o master para la versión estable, develop para integrar los avances y ramas feature/ para desarrollar funcionalidades específicas. Una vez revisados los cambios, estos se integran en develop y, cuando se dispone de una versión estable, se incorporan a la rama principal.

La integración de ambos diseños permite representar tanto la organización interna del sistema como el proceso de control de versiones empleado para mantener la documentación y los componentes del proyecto. Esta estructura favorece la separación de responsabilidades, la trazabilidad de los cambios y la posibilidad de ampliar el sistema en el futuro.

Aplicación de patrones de diseño:

¿QUÉ SON LOS PATRONES DE DISEÑO?

Los patrones de diseño son soluciones reutilizables para problemas frecuentes que aparecen al organizar un sistema de software. No son programas completos que se copian y ejecutan automáticamente, sino formas de estructurar las clases y sus relaciones para que el código sea más comprensible, reutilizable y fácil de mantener.

¿QUÉ PATRONES UTILIZAREMOS?

1. Singleton
   - Mantiene un único punto de acceso a la instancia central del gestor de casos.

Se utilizará en la clase GestorCasos, encargada de coordinar el registro de los casos de rescate, la asignación de voluntarios y la actualización de los estados de los animales.

2. Observer
   - Permite notificar a los voluntarios interesados cuando se registra un nuevo rescate.

Cuando un ciudadano registre un nuevo caso de rescate, el sistema podrá notificar a los voluntarios y veterinarios para que conozcan la situación y puedan actuar.

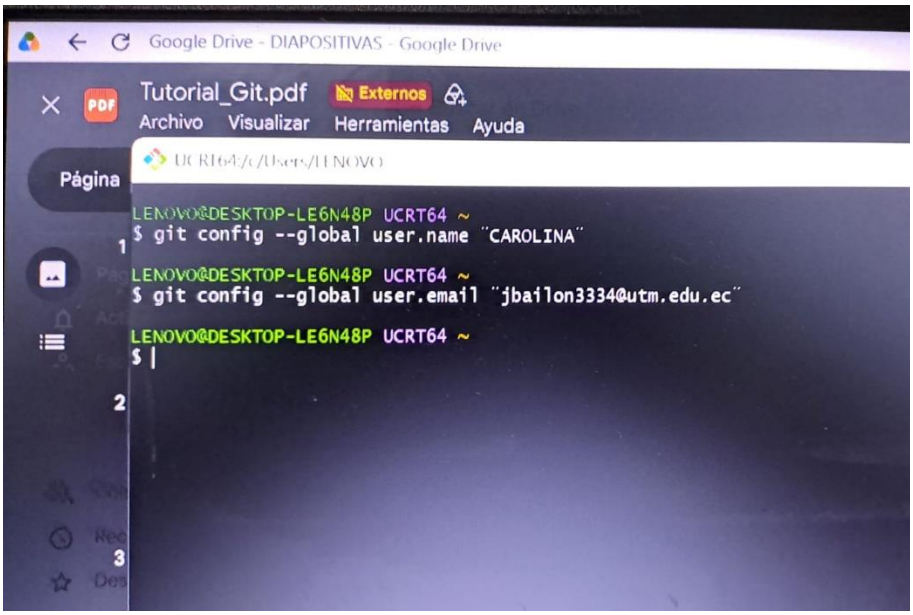
3. Factory Method
   - Organiza la creación de expedientes de perros, gatos y fauna silvestre.

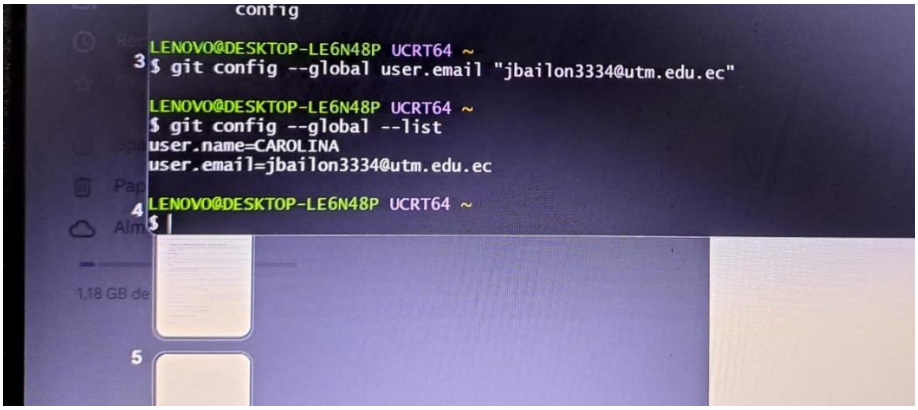
Se utilizará para crear expedientes específicos según la especie del animal rescatado, como perros, gatos y fauna silvestre.

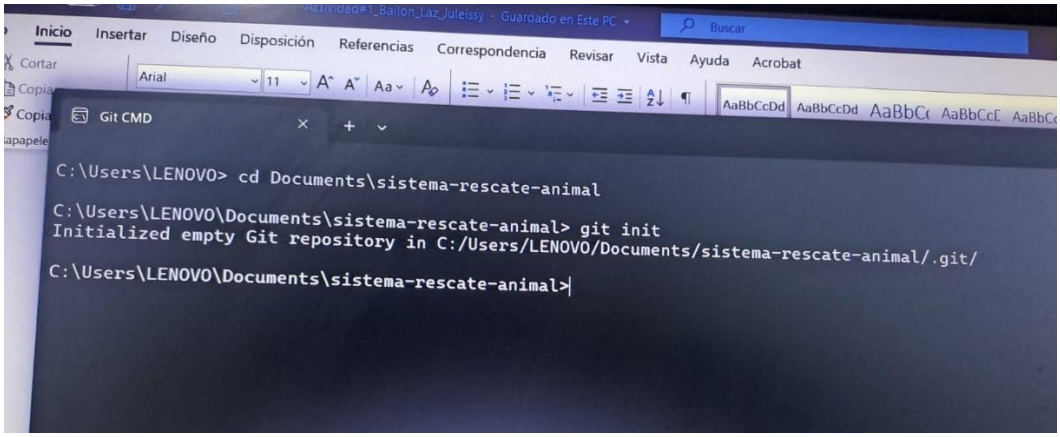
4. MVC (Modelo-Vista-Controlador)

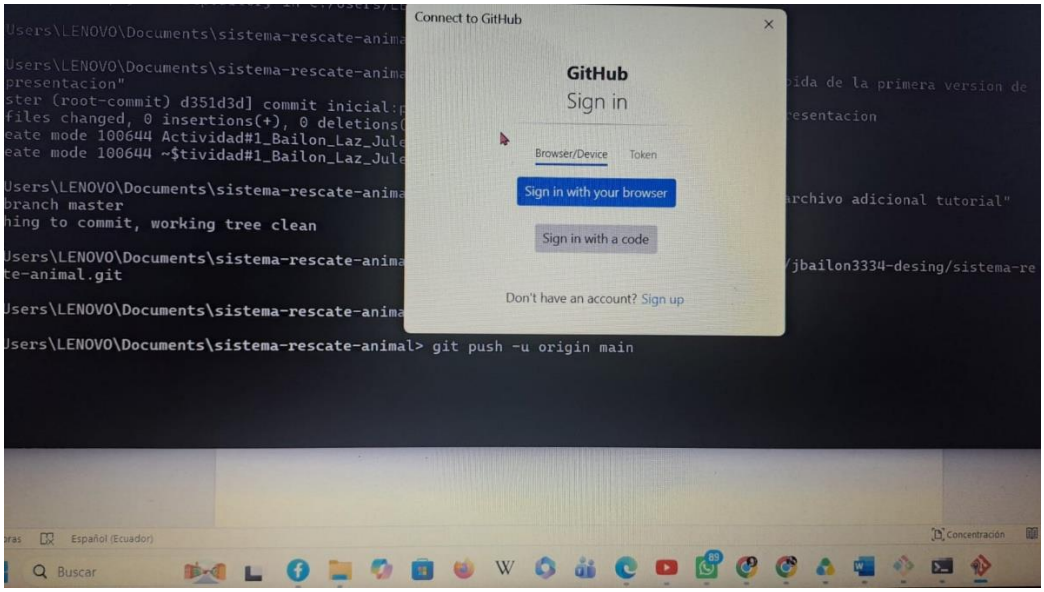
   - Separa la interfaz, el procesamiento de solicitudes y los datos y reglas del sistema.
     - Modelo: representa los casos de rescate, los expedientes de los animales y las reglas del sistema.
     - Vista: contiene los formularios de registro, los listados de casos y las pantallas de seguimiento.
     - Controlador: recibe las acciones del usuario, valida las solicitudes y coordina las operaciones correspondientes.

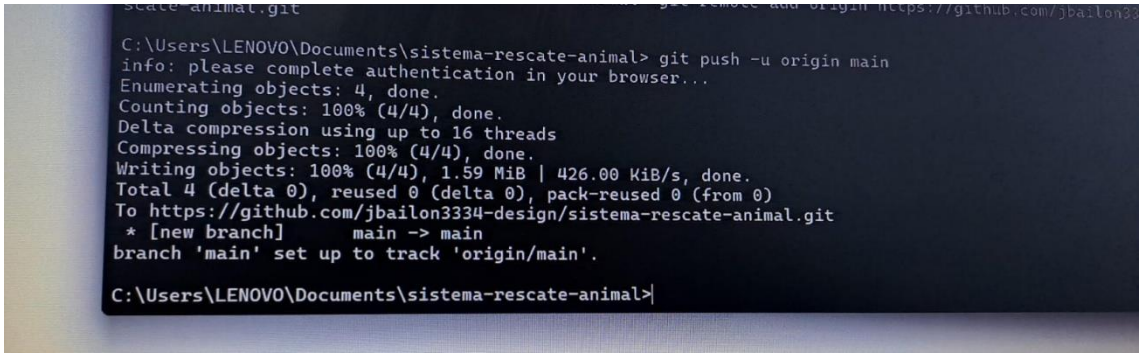
Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.








Connect to GitHub
GitHub
Sign in
Browser/Device
Token
Sign in with your browser
Sign in with a code
Don't have an account? Sign up
Users\LENOVO\Documents\sistema-rescate-animal> git push -u origin main
Español (Ecuador)
Buscar


C:\Users\LENOVO\Documents\sistema-rescate-animal> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.59 MiB | 426.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jbailon3334-design/sistema-rescate-animal.git
 * [new branch]   main -> main
branch 'main' set up to track 'origin/main'.
C:\Users\LENOVO\Documents\sistema-rescate-animal>

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)

A. Shvets, Sumergete en los patrones de diseño. 2019.

https://www.drawio.com/docs/manual/new-diagram/

https://lucid.co/es/lucidchart

https://github.com/?locale=es-419